

Task No: 9

Date: 24/9/25

Implement exceptions and
Exceptional handling in python.

Aim: To implement exceptions and exceptional handling in python

Prob:

Problem 9.1. you are developing a python program that processes a list of students grades. The program is designed to allow the user to select a grade by specifying an index number. However, you need to ensure that the program handles cases where the user inputs an index that is out of range. i.e., an index does not exist in the list.

Algorithm:

1. Start the program.
2. Initialise a list of grades (e.g., [85, 90, 78, 92, 88]).
3. Prompts the user to enter the index of the grade they wish to view.
4. Attempts to display the grade at the specified index.
5. If the index is out of range, catches the Index Error and prints an error message.
"Invalid index. please enter a valid index".

Program:

```
# initialize the list of grades
grades = [85, 90, 78, 92, 88].

# Display the grades list.
print("grades list:", grades)

try:
    index = int(input("Enter the index of the grade you want to view: "))
    print(f"The grade at index {index} is: {grades[index]}")
except IndexError:
    print("Invalid Index. please enter a valid index.")
except ValueError:
```


Output:

Grades list : [85, 90, 78, 92, 88]

Enter the Index of the grade you want to view: 10

Invalid index. please enter a valid index.

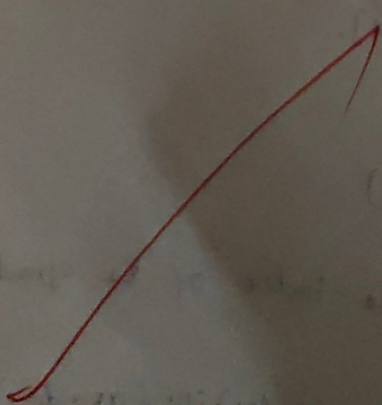
output:

Enter the numerator: 10

Enter the denominator: 0

Error:

Error: Division by zero is not allowed.



`print("Invalid input. please enter a numerical index")`.

Date: 24/9/25

Problem 9.2 you are developing a python calculator program that performs basic arithmetic operations. one of the key functionalities is to divide two numbers entered by the user. However, dividing by zero is not allowed and would cause program to crash if not handled properly.

Algorithm:-

1. start the program.
2. prompts the user to enter the two numbers: a numerator and a denominator.
3. Attempts to divide the numerator by the denominator.
4. If the denominator is zero, catches the `ZeroDivisionError` and displays an error message. "Error: Division by zero is not allowed".

Program:

Function to perform division.

`def divide - numbers():`

`try:`

`# Prompt the user to enter numerator.`

`numerator = float(input("Enter the numerator: "))`

`denominator = float(input("Enter the denominator: "))`

`result = numerator / denominator`

`print(f"result: {result}")`

`except ZeroDivisionError:`

`print("Error: Division by zero is not allowed")`

`except value error:`

`print("Error: please enter valid numbers")`

`divide - numbers()`

output

Enter a number > 15

Exception occurred: Invalid Age.

Date: 24/9/25
 Problem 9.3: you are building a python application to determine if a person is eligible to vote based on their age. According to the rules, only individuals who are 18 years or older are allowed to vote. To enforce this rule, you decide to create a custom exception called Invalid Age Exception which will be raised whenever an age below 18 is entered.

Algorithm

1. Define the custom exception.
2. Prompt the user for input.
3. Check if the age is below 18.
4. Raise an exception if the condition is met.
5. Handle the exception with a custom error message.

Program

Class Invalid Age Exception (Exception):

"Raised when the input value is less than 18".
 Pass.

number = 18

```
try:
    input_num = int(input("Enter a number:"))
    if input_num < number:
        raise Invalid Age Exception
    else:
        print("Eligible to vote")
except Invalid Age Exception:
    print("Exception occurred. Invalid Age")
```

VEL TECH	
EX NO.	9
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	
TOTAL (20)	15
SIGNATURE DATE	

Result - Thus, the program for implement exceptions and exceptional handling is executed and verified successfully.